Terraform Interview Questions & Answers

1. What is Terraform?
Terraform Interview Answer
Interviewer: What is Terraform?
Me: Terraform is an Infrastructure as Code tool developed by HashiCorp. We use it to create, manage, and provision infrastructure using configuration files instead of manually creating resources through the cloud console.
For example, if I need AWS infrastructure like VPC, EC2, RDS, Load Balancer, and S3, I can define them in Terraform code. Terraform communicates with AWS through the AWS provider and creates or manages those resources.
The main benefit is that our infrastructure becomes repeatable, version-controlled, and easier to manage.

---

2. What is a Terraform Variable?
Terraform Variable Interview Answer
Interviewer: What is a Terraform variable and why do you use it?
Me: Terraform variable is basically an input to our Terraform configuration.
We use variables because we don't want to hard-code values directly inside our resources.
For example, if I have an EC2 instance, Dev might need t3.micro and Prod might need t3.large. Instead of changing the Terraform code for every environment, I define instance_type as a variable and pass the required value.
So variables help us avoid hard-coding and make the same Terraform configuration reusable with different values.
The flow is basically: define the variable, provide a value, and use that variable inside the resource with var.variable_name.

---

3. What is terraform.tfvars?
Terraform tfvars Interview Answer
Interviewer: What is terraform.tfvars?
Me: terraform.tfvars is a file where we provide actual values for the variables defined in our Terraform configuration.
For example, if I define an instance_type variable, I can provide the value like t3.micro in terraform.tfvars.
So variables.tf defines what inputs Terraform expects, and terraform.tfvars provides the actual values for those inputs.
This is useful because we can keep our Terraform code generic and provide different values for different environments.

---

4. Difference between variables.tf and terraform.tfvars?
Variables vs tfvars
Interviewer: What is the difference between variables.tf and terraform.tfvars?
Me: variables.tf is used to declare the variables and define things like their type, default value, or validation.
terraform.tfvars is used to provide the actual values for those variables.
For example, in variables.tf I can say that I need an instance_type variable of type string.
Then in terraform.tfvars, I can provide instance_type = "t3.micro".
So simply, variables.tf tells Terraform **what input I need**, while terraform.tfvars tells Terraform **what value to use**.

---

5. What is a Terraform Module?
Terraform Module Interview Answer
Interviewer: What is a Terraform module?
Me: A Terraform module is a collection of Terraform configuration files that we can reuse.
For example, suppose my company has a standard way of creating an EC2 server. Instead of writing the same EC2 configuration separately for Dev, Stage, and Prod, I can create an EC2 module once and reuse it.
The environment-specific configuration passes inputs to the module, such as instance type or instance count.
So basically, modules help us organize Terraform code and avoid duplication by making infrastructure code reusable.

---

6. Why do you use modules?
Why Terraform Modules
Interviewer: Why do you use Terraform modules?
Me: I use modules mainly for reusability, consistency, and better organization.
For example, if Dev, Stage, and Prod all need the same type of VPC, I don't want to maintain three copies of the VPC code.
I create one VPC module and use it from all three environments with different input values.
That way, common infrastructure logic stays in one place and environment-specific values are passed as inputs.

---

7. What are Terraform Data Sources?
Terraform Data Source Interview Answer
Interviewer: What is a Terraform data source?
Me: A data source is used to read information about existing infrastructure or existing resources.
For example, suppose a VPC is already created in AWS and I don't want Terraform to create a new VPC. But I need its VPC ID to create another resource.
I can use a data source to retrieve that existing VPC information and use the value in my Terraform configuration.
So the main difference is, a resource is generally used to create or manage infrastructure, while a data source is used to read existing information.

---

8. Resource vs Data Source?
Resource vs Data Source
Interviewer: What is the difference between a resource and a data source?
Me: A resource is used when Terraform needs to manage an infrastructure object.
For example, if I write an aws_instance resource, Terraform can create and manage that EC2 instance.
A data source is used when the resource already exists and I only need to retrieve information about it.

So I remember it as: **resource manages infrastructure, data source reads existing infrastructure information.**

---

9. What is Terraform Workspace?
Terraform Workspace Interview Answer
Interviewer: What is a Terraform workspace?
Me: A Terraform workspace allows us to use the same Terraform configuration with separate state data.
For example, if I use the same configuration for Dev and Stage, I can have a Dev workspace and a Stage workspace, and Terraform maintains separate state for each workspace.
So the important point is that workspaces separate the Terraform state while allowing the configuration to remain the same.
However, for larger environments, teams often use separate environment directories or separate state configurations instead of relying only on workspaces.

---

10. Why do you use Terraform Workspaces?
Why Terraform Workspaces
Interviewer: Why would you use Terraform workspaces?
Me: I use workspaces when I want to maintain separate state for the same Terraform configuration.
For example, the same configuration might be used for multiple isolated environments, and each workspace keeps its own state.
This prevents Terraform from treating all those environments as one state.
But I would choose the approach based on the project architecture, because workspaces are not always the best way to represent completely different environments.

---

11. What is Terraform State?
Terraform State Interview Answer
Interviewer: What is Terraform state?
Me: Terraform state is the information Terraform maintains about the infrastructure it manages.
For example, if Terraform creates an EC2 instance, Terraform needs to know that the instance exists and which real AWS resource corresponds to the Terraform resource.
Terraform stores this information in the state.
During the next plan or apply, Terraform compares the configuration, the state, and the real infrastructure to determine what changes are required.
So state is very important because Terraform uses it to track and manage infrastructure.

---

12. What is a Remote Backend?
Terraform Remote Backend Interview Answer
Interviewer: What is a Terraform remote backend?
Me: A remote backend is a place where Terraform stores its state remotely instead of keeping the state only on the local machine.
For example, in AWS we can use an S3 bucket as a backend for Terraform state.
This is useful in a team because multiple engineers and CI/CD systems need access to the same state.
A remote backend also allows us to configure additional state-management features such as locking, depending on the backend and setup.
So instead of having the state only on my laptop, the team can work with a centrally managed state.

---

13. Why do you need a Remote Backend?
Why Remote Backend
Interviewer: Why do you use a remote backend?
Me: In a real project, Terraform is usually used by multiple engineers and CI/CD pipelines.
If the state is stored only on my laptop, other team members don't have the same state and it can create synchronization problems.
With a remote backend, the state is stored centrally and can be accessed by the authorized team or CI/CD system.
It also gives us better control over state security and collaboration.

---

14. What are Terraform Provisioners?
Terraform Provisioners Interview Answer
Interviewer: What are Terraform provisioners?
Me: Provisioners allow Terraform to execute scripts or commands during resource creation or destruction.
For example, after creating an EC2 instance, a remote-exec provisioner can execute commands on that server.
There are different provisioners such as local-exec and remote-exec.
However, provisioners are generally considered a last resort. If possible, I prefer cloud-init, user data, configuration management tools, or other purpose-built solutions because they are usually more predictable and maintainable.

---

15. Difference between local-exec and remote-exec?
local-exec vs remote-exec
Interviewer: What is the difference between local-exec and remote-exec?
Me: local-exec runs a command on the machine where Terraform itself is running.
For example, if Terraform is running on a CI/CD server, the command runs on that CI/CD server.
remote-exec runs commands on the remote resource, such as an EC2 instance, after Terraform connects to it.
So simply, **local-exec runs locally, remote-exec runs on the remote machine.**

---

16. What is Terraform DR / Failover?
Terraform DR Failover Interview Answer
Interviewer: How can Terraform be used for Disaster Recovery?
Me: Terraform can be used to define and recreate our infrastructure in a secondary region or environment.

For example, suppose our application is running in AWS Region A. We can use Terraform to define the required infrastructure in Region B as a disaster recovery environment.
If Region A becomes unavailable, traffic can be redirected to the secondary environment using services such as DNS or traffic-routing mechanisms.
Terraform's role is mainly to provision and manage the infrastructure. The actual traffic failover is handled by the appropriate networking or DNS components.
So Terraform helps us make the DR infrastructure reproducible through code.

---

17. What is HashiCorp Vault?
Terraform Vault Interview Answer
Interviewer: What is HashiCorp Vault and why would you use it with Terraform?
Me: HashiCorp Vault is a secrets-management system.
We can use it to securely manage sensitive information such as database passwords, API credentials, and tokens instead of hard-coding them in Terraform files.
Terraform can retrieve required secrets from Vault and use them when configuring infrastructure.
However, I would also make sure that Terraform state is properly secured, because using Vault does not automatically mean that a secret can never appear in Terraform state.
So Vault is mainly used for **secure secret management**, while the Terraform backend is responsible for managing Terraform state.

---

18. Provider ante enti?
Terraform Provider Interview Answer
Interviewer: What is a Terraform provider?
Me: A provider is the component that allows Terraform to communicate with an external platform or service.
For example, if I'm managing AWS infrastructure, I use the AWS provider.
The provider understands how to communicate with the AWS APIs and exposes resources and data sources that Terraform can use.
So Terraform itself is the infrastructure-as-code engine, and the provider acts as the integration with AWS or another platform.

---

19. terraform init ante enti?
Terraform Init Interview Answer
Interviewer: What does terraform init do?
Me: terraform init initializes the Terraform working directory.
It downloads and prepares the required providers and modules, and it also initializes the configured backend.
I normally run terraform init when I start working with a Terraform configuration or when provider, module, or backend configuration changes require reinitialization.

---

20. terraform plan ante enti?
Terraform Plan Interview Answer
Interviewer: What does terraform plan do?
Me: terraform plan shows what Terraform intends to change without actually applying those changes.
Terraform compares the desired configuration with the current state and the existing infrastructure, then shows the proposed actions such as create, update, or destroy.
I use the plan to review the changes before running terraform apply.

---

21. terraform apply ante enti?
Terraform Apply Interview Answer
Interviewer: What does terraform apply do?
Me: terraform apply executes the changes proposed by Terraform.
For example, if the plan shows that one EC2 instance needs to be created, terraform apply will make that change against AWS.
Before applying in a production environment, I would normally review the plan carefully and follow the team's approval process.

---

22. terraform destroy ante enti?
Terraform Destroy Interview Answer
Interviewer: What does terraform destroy do?
Me: terraform destroy tells Terraform to remove the resources that are managed by that Terraform configuration and state.
For example, if Terraform created an EC2 instance and other resources for a temporary environment, destroy can remove those managed resources.
I would be very careful with this command, especially in production, because it can delete infrastructure.

⭐ **Interview lo ee concepts madhya connection**
Terraform
↓
Provider
↓
AWS tho communicate chestundi
↓
Resources / Data Sources
↓
Infrastructure
↓
Variables
↓

Inputs control chestayi
↓
Modules
↓
Code reuse chestayi
↓
State
↓
Terraform infrastructure ni track chestundi
↓
Remote Backend
↓
State ni centrally store chestundi
↓
Workspace
↓
Separate state instances
↓
Provisioner
↓
Commands/scripts (last resort)
↓
Vault
↓
Secrets management
↓
DR
↓
Secondary infrastructure ni code tho reproduce/manage cheyyadam
Interview preparation order: Terraform → Provider → Resource → Variable → tfvars → Data Source → Module → State → Backend → Workspace → Provisioner → Vault → DR/Failover. Ee order lo prepare aithe concepts okadaniki okati connect avuthayi

---

1. **Terraform Variables**
2. **Terraform Workspaces**
3. **Terraform Modules**
4. **Terraform Provisioners**
5. **Terraform Remote Backend**
6. **Terraform Data Sources**
7. **Terraform DR / Failover**
8. **Terraform + HashiCorp Vault**

1. Terraform Variables
Interview Q&A — Variables
Interviewer: What are Terraform variables? Why do you use them?
You: Terraform variables are basically inputs to our Terraform configuration. We use them when some values can change between environments or deployments.
For example, I'm creating an EC2 instance and the instance type is different in Dev and Prod. Instead of hard-coding t3.micro or t3.large directly in my resource, I define instance_type as a variable and pass the required value.
For example, Dev can use t3.micro and Prod can use t3.large, while the actual Terraform resource code stays the same.
So basically, I use variables to avoid hard-coding and to make my Terraform code reusable and flexible.
Interviewer: Where do you define the variable?
You: Usually I define the variable in variables.tf, provide its value through a .tfvars file or another supported input method, and reference it in the configuration using var.variable_name.
Interviewer: Give me an example.
You: For example, I can define variable "instance_type", then provide instance_type = "t3.micro" for Dev. In my EC2 resource, I use instance_type = var.instance_type. Terraform then uses the value I provided when creating the instance.

---

2. Terraform Workspaces
Interview Q&A — Workspaces
Interviewer: What are Terraform workspaces?
You: Terraform workspaces allow us to maintain separate state for the same Terraform configuration.
For example, if I have one Terraform configuration and I want to use it for Dev and Stage, I can have a dev workspace and a stage workspace. Each workspace has its own state, so Terraform keeps track of those environments separately.
Interviewer: Why would you use workspaces?

You: I would use them when the same configuration needs isolated state. It prevents different environments from being tracked in the same state.
Interviewer: Does a workspace create a separate AWS account or region?
You: No. A workspace itself doesn't create an AWS account, region, or infrastructure. Its main purpose is to maintain a separate Terraform state for that workspace.
Interviewer: Do you always use workspaces for Dev and Prod?
You: Not necessarily. In larger projects, teams often use separate environment directories and separate state configurations because they provide clearer isolation and control. I choose the approach based on the project architecture.

---

3. Terraform Modules
Interview Q&A — Modules
Interviewer: What is a Terraform module?
You: A Terraform module is a reusable collection of Terraform configuration.
For example, suppose our company has a standard way of creating an EC2 server. Instead of writing the same EC2 code separately for Dev, Stage, and Prod, I can create an EC2 module once and reuse it.
The module contains the actual infrastructure logic, and the environment passes values such as instance type, instance count, or other configuration through variables.
Interviewer: Why do you use modules?
You: Mainly for reusability and to avoid duplicate code. They also help us maintain a consistent infrastructure design across environments.
Interviewer: How do you call a module?
You: I use a module block and specify the module's source.
For example:
module "ec2" { source = "../../modules/ec2" }
Then I can pass required variables to that module.
Interviewer: Give me a real example.
You: In a Netflix-type application, I could have an ec2 module containing the common EC2 configuration. Dev can call that module with t3.micro, while Prod can call the same module with t3.large. The module code remains common and only the inputs change.

---

4. Terraform Provisioners
Interview Q&A — Provisioners
Interviewer: What are Terraform provisioners?
You: Provisioners are used to execute commands or scripts during resource creation or destruction.
For example, suppose Terraform creates an EC2 instance and I need to execute a command on that server after creation. I can use a remote-exec provisioner.
Interviewer: What types of provisioners do you know?
You: Two commonly used ones are local-exec and remote-exec.
local-exec runs the command on the machine where Terraform is running.
remote-exec runs commands on the remote resource, such as an EC2 instance.
Interviewer: Do you use provisioners regularly?
You: No. I consider provisioners a last resort because they can make infrastructure provisioning less predictable. Whenever possible, I prefer user data, cloud-init, configuration-management tools, or other purpose-built mechanisms.
Interviewer: Give me an example.
You: For example, after creating an EC2 instance, I could use remote-exec to run an installation command. But if the same setup can be done using EC2 user data or cloud-init, I would generally prefer that approach.

---

5. Terraform Remote Backend
Interview Q&A — Remote Backend
Interviewer: What is a Terraform backend?
You: A Terraform backend determines where Terraform stores its state and how that state is managed.
By default, Terraform can keep the state locally. But in a real company, multiple engineers and CI/CD pipelines may work on the same infrastructure, so keeping the state only on one laptop is not practical.
That's why we normally use a remote backend.
Interviewer: Give me an example.
You: In AWS, we can use an S3-based backend to store Terraform state remotely.
So instead of:
Developer laptop → terraform.tfstate
we have:
Developer / CI-CD → Remote backend → Terraform state
Interviewer: Why is remote state important?
You: It gives the team a centralized location for state and makes collaboration easier. It also allows us to implement appropriate security and concurrency controls.
Interviewer: Is backend the same as state?
You: No. State is the information Terraform uses to track managed infrastructure. Backend is the mechanism and location used to store and manage that state.

---

6. Terraform Data Sources
Interview Q&A — Data Sources
Interviewer: What is a Terraform data source?
You: A data source is used to read information about an existing resource or external data without creating that resource through the data source.

For example, suppose a VPC already exists in AWS, but my Terraform configuration needs its VPC ID. Instead of creating another VPC, I can use an AWS VPC data source to retrieve the existing VPC information.
Interviewer: What's the difference between a resource and a data source?
You: A resource is used by Terraform to create or manage infrastructure.
A data source is used to read information that already exists.
So if I need to create a VPC, I use a resource. If the VPC already exists and I only need its information, I use a data source.
Interviewer: Give me a practical example.
You: For example, the networking team might already have created a VPC. My application Terraform code can use a data source to retrieve that VPC and then deploy application resources using the existing network instead of creating another VPC.

---

7. Terraform DR / Failover
Interview Q&A — DR and Failover
Interviewer: How can Terraform help with Disaster Recovery?
You: Terraform helps us define our infrastructure as code, so we can reproduce the required infrastructure in a secondary region or environment.
For example, suppose our application is running in AWS Region A. We can define the required DR infrastructure in Region B using Terraform.
If Region A becomes unavailable, the application can be brought up or traffic can be redirected to Region B depending on how the DR architecture is designed.
Interviewer: Does Terraform itself perform the failover?
You: Not usually. Terraform is mainly responsible for provisioning and managing the infrastructure.
The actual traffic failover can be handled by services such as Route 53, load balancers, or other traffic-routing mechanisms.
Interviewer: So what's Terraform's role in DR?
You: Terraform makes the DR infrastructure reproducible and manageable as code. Instead of manually rebuilding servers, networking, databases, and other infrastructure, we can define them in Terraform and provision the required environment consistently.
Interviewer: What would you consider while designing DR?
You: I would consider the required RTO and RPO, data replication, application dependencies, networking, DNS or traffic routing, backup strategy, and how the secondary infrastructure will be tested and maintained.

---

8. Terraform + HashiCorp Vault
Interview Q&A — Terraform and Vault
Interviewer: What is HashiCorp Vault and how can you use it with Terraform?
You: HashiCorp Vault is a secrets-management solution. It is used to securely store and manage sensitive information such as database passwords, API credentials, and tokens.
Instead of hard-coding those credentials directly in Terraform files, Terraform can retrieve required secrets from Vault.
Interviewer: Why don't you put the password directly in Terraform?
You: Because Terraform code is normally stored in version control, and hard-coding passwords creates a security risk. I prefer keeping secrets in a dedicated secrets-management system such as Vault.
Interviewer: How does the flow work?
You: The basic flow is:
Terraform needs a secret → Terraform authenticates to Vault → Vault provides the required secret → Terraform uses it to configure the required resource.
Interviewer: Does using Vault guarantee the secret won't be stored in Terraform state?
You: No. That's an important point. If Terraform uses a secret as part of a resource configuration, the value may be stored in Terraform state depending on the resource and configuration.
So I need to secure both sides: Vault for secret management and the Terraform backend and state for state security.
Interviewer: Give me a real example.
You: For example, if an application needs an RDS database password, I can store the password in Vault instead of putting it directly into a Terraform file. Terraform can retrieve the required secret and use it while configuring the database.

DEEP EXPLANATION WITH EXAMPLES

1. Terraform Variables
Variable ante enti?
Terraform Variable ante **Terraform code ki values bayata nunchi ivvadaniki use chesedi**.
Simple example Netflix application ki EC2 server create chestunnav.
EC2 ki instance_type kavali.
Direct ga code lo:

```
resource "aws_instance" "netflix" {
  instance_type = "t3.micro"
}
```

ani raaste t3.micro hard-coded aipoyindi.
Repu Production lo t3.large kavali ante main.tf code marchali.
Anduke variable.

Step 1: variables.tf

```
variable "instance_type" {
  type = string
}
```

Idi Terraform ki cheppedi:
"Naa infrastructure ki instance_type ane input value kavali."
Ikkada actual value t3.micro ivvaledu.
Just **input ni define** chesam.

Step 2: terraform.tfvars
instance_type = "t3.micro"
Ikkada actual value icham.
So:
variables.tf
↓
"instance_type ane input kavali"

terraform.tfvars
↓
"aa input value t3.micro"

Step 3: main.tf

```
resource "aws_instance" "netflix" {
  instance_type = var.instance_type

  tags = {
    Name = "netflix-server"
  }
}
```

var.instance_type ante:
variables lo unna instance_type value ni ikkada use cheyyi.
Final ga:
terraform.tfvars
↓
t3.micro
↓
var.instance_type
↓
EC2
↓
t3.micro

Netflix lo enduku useful?
Same infrastructure Dev, Stage, Prod lo use cheyyali.
DEV
↓
instance_type = t3.micro

STAGE
↓
instance_type = t3.medium

PROD
↓
instance_type = t3.large
EC2 code same:

```
resource "aws_instance" "netflix" {
  instance_type = var.instance_type
}
```

Code same → value different.
That's the main purpose of variable.

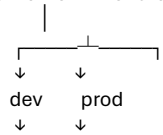
---

2. Terraform Workspaces
Workspace ardham avvalante mundu **Terraform State** ardham avvali.
Terraform AWS lo EC2 create chesindi anuko.
Terraform ki:
"Nenu ee EC2 ni create chesanu, idi naa management lo undi."
ani track cheyyali.
Aa information ni Terraform **state** lo maintain chestundi.
AWS
↓
Netflix EC2

Terraform
↓

State
↓
"Ee EC2 naa management lo undi"

Workspace ante enti?
Workspace ante **same Terraform configuration ki separate state maintain cheyyadam**.
Example:
Same Netflix Terraform Code



Dev state lo Dev resources information untundi.
Prod state lo Prod resources information untundi.

Commands
Dev workspace create:
terraform workspace new dev
Prod:
terraform workspace new prod
Dev ki switch:
terraform workspace select dev
Prod ki:
terraform workspace select prod

Netflix example
Same Terraform code:

```
resource "aws_instance" "netflix" {
  instance_type = var.instance_type
}
```

Dev workspace:
dev workspace
↓
Dev state
↓
Dev EC2
Prod workspace:
prod workspace
↓
Prod state
↓
Prod EC2
Important point
Workspace **AWS lo Dev/Prod ni magically create cheyyadu**.
Workspace main purpose:
Same configuration
↓
Separate Terraform state
Large real-world projects lo separate environment directories/accounts/state configurations use cheyyadam kuda common.

---

3. Terraform Modules
Module ante enti?
Module ante **reusable Terraform code**.
Netflix infrastructure lo:
VPC
EC2
RDS
ALB
S3
unnayi.
Suppose EC2 create cheyyadaniki 30 lines Terraform code undi.
Dev kosam 30 lines.
Stage kosam same 30 lines.
Prod kosam same 30 lines.
Copy-paste chesthe:
Dev → EC2 code
Stage → EC2 code
Prod → EC2 code
Same code 3 places.

Instead module create chestham.

```
modules/
  ↓
 ec2/
  ├── main.tf
  ├── variables.tf
  └── outputs.tf
```

Module lo actual code

modules/ec2/main.tf

```
resource "aws_instance" "netflix" {

 ami         = var.ami_id
 instance_type = var.instance_type

 tags = {
  Name       = "netflix-server"
  Environment = var.environment
 }
}
```

modules/ec2/variables.tf

```
variable "ami_id" {
 type = string
}

variable "instance_type" {
 type = string
}

variable "environment" {
 type = string
}
```

Ippudu idi reusable EC2 module.

Dev module ni use chestundi

environment/dev/main.tf

```
module "netflix_ec2" {

 source = "../../modules/ec2"

 ami_id      = "ami-xxxxxxxx"
 instance_type = "t3.micro"
 environment   = "dev"
}
```

Meaning:

```
Dev
 ↓
EC2 module ni use cheyyi
 ↓
instance_type = t3.micro
```

Prod same module ni use chestundi

```
module "netflix_ec2" {

 source = "../../modules/ec2"

 ami_id      = "ami-xxxxxxxx"
 instance_type = "t3.large"
 environment   = "prod"
}
```

Meaning:

```
Prod
 ↓
Same EC2 module
 ↓
instance_type = t3.large
```

Actual EC2 creation logic:

modules/ec2/main.tf



↓ ↓
DEV PROD
t3.micro t3.large
Module = common code.

---

4. Terraform Provisioners
Provisioner ante enti?
Provisioner ante **resource create ayyaka command/script execute cheyyadaniki use chesedi**.
Netflix server create chestunnav.
Terraform
↓
EC2 create
↓
EC2 ready
↓
Some command execute
Example:
EC2 create ayyaka nginx install cheyyali.

```
provisioner "remote-exec" {

 inline = [
  "sudo apt update",
  "sudo apt install nginx -y"
 ]
}
```

Meaning:
EC2 create ayyaka aa EC2 machine lo ee commands run cheyyi.

---

local-exec
Terraform run chestunna machine lo command run chestundi.

```
provisioner "local-exec" {
 command = "echo Netflix EC2 created"
}
```

Flow:
Terraform machine
↓
echo command

---

remote-exec
Remote EC2 machine lo command run chestundi.
Terraform
↓
EC2
↓
Command

Netflix example
Netflix EC2
↓
Server create
↓
Nginx install
↓
Application setup
Provisioner tho cheyyachu.
Kani important point:
Provisioners ni generally last resort ga use chestham.
Possible ayithe:
User Data
Cloud-init
Configuration Management
Pre-built Machine Image
lanti methods prefer chestham.
Endukante provisioners Terraform dependency/ordering and failure handling ni complex cheyyagalavu.

---

5. Terraform Remote Backend
First Backend ante enti?
Terraform ki state kavali.
Default/local setup lo:
Developer Laptop
↓

terraform.tfstate
Problem enti?
Netflix project lo:
Developer A
Developer B
Developer C
CI/CD
andaroo Terraform run cheyyachu.
State okka developer laptop lo unte migatha vallaki same state access undadu.
Anduke remote backend.

Remote Backend ante?
Terraform state ni remote location lo store cheyyadam.
AWS lo example:



Netflix example

```
terraform {
  backend "s3" {
    bucket = "netflix-terraform-state"
    key    = "prod/terraform.tfstate"
    region = "ap-south-1"
  }
}
```

Meaning:
Terraform
↓
State
↓
AWS S3
↓
netflix-terraform-state

Enduku?
Team members andariki centralized state kavali.
Remote Backend
↓
Central state
↓
Developers + CI/CD
↓
Same infrastructure state
State ni secure cheyyadam kuda important.
Modern AWS S3 backend setups lo locking/concurrency control backend capabilities/configuration batti handle chestaru.

---

6. Terraform Data Sources
Data Source ante enti?
Data Source ante:
Already AWS lo unna resource information ni Terraform read cheyyadam.
Suppose Netflix company VPC already create chesindi.
AWS
↓
Existing VPC
↓
vpc-123456
Nuvvu malli VPC create cheyyakudadhu.
Kani aa VPC information Terraform ki kavali.
Appudu Data Source.

Example

```
data "aws_vpc" "netflix" {
  id = "vpc-123456"
}
```

Idi VPC create cheyyadu.
Terraform AWS ki query chesi existing VPC details read chestundi.

Resource vs Data Source
Idi very important:
RESOURCE
↓
Create / Manage
DATA SOURCE
↓
Read existing
Example:

```
resource "aws_vpc" "netflix" {
  cidr_block = "10.0.0.0/16"
}
```

Idi VPC create/manage cheyyadaniki.
But:

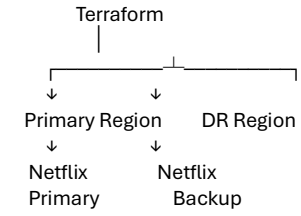
```
data "aws_vpc" "netflix" {
  id = "vpc-123456"
}
```

Idi already existing VPC ni read cheyyadaniki.

Netflix situation
Network team already VPC create chesindi.
Application team Netflix EC2 create chestundi.
Application team:
Existing VPC
↓
Data Source
↓
VPC information
↓
Netflix infrastructure
So existing resource ni recreate cheyyakunda information reuse cheyyachu.

---

7. Terraform DR / Failover
DR ante enti?
DR = **Disaster Recovery**.
Suppose Netflix application:
AWS Mumbai
↓
Netflix Production
Mumbai region lo major issue vachindi anuko.
Secondary region ready undali:
AWS Hyderabad/another region
↓
Netflix DR
Conceptually:



Terraform role enti?
Terraform tho:
VPC
Subnet
EC2
ALB
RDS-related infrastructure
Security Groups
Other infrastructure
secondary region lo define/manage cheyyachu.
So:
Terraform
↓
DR Infrastructure
↓
Secondary AWS Region

Failover ante?
Primary region down ayindi.
Users
↓
Traffic Routing
↓
Primary ❌
Traffic secondary region ki vellali:
Users
↓
DNS / Traffic Routing
↓
DR Region ✅
↓
Netflix
Important:
Terraform infrastructure prepare/manage chestundi.
Traffic ni switch cheyyadam DNS/traffic-routing system responsibility.
So DR and Failover rendu exact same kaavu.
DR
↓
Backup infrastructure/recovery capability

Failover
↓
Traffic ni healthy environment ki switch cheyyadam

---

8. Terraform + HashiCorp Vault
Vault ante enti?
HashiCorp Vault ante **secrets ni securely store and manage cheyyadaniki system**.
Netflix ki RDS database undi.
Database ki:
Username
Password
kavali.
Password ni Terraform code lo:
password = "Netflix@123"
ani rayadam manchidi kaadu.
Enduku?
Terraform code Git lo undachu.
Terraform Code
↓
Git Repository
↓
Password exposed risk

Vault use chesthe
Password Vault lo store chestham.
Vault
↓
Netflix DB Password
Terraform required time lo Vault nunchi secret retrieve/use cheyyachu.
Concept:
Vault
↓
DB Password
↓
Terraform
↓
RDS

Vault lo em store cheyyachu?
DB passwords
API keys
Tokens
Credentials
Certificates
Other secrets

Important point

Vault use chesthe:
"Secret Terraform state lo eppudu undadu"
ani guarantee kaadu.
Suppose:
Vault
↓
Password
↓
Terraform resource argument
↓
Terraform state
Secret resource argument lo use chesthe state lo sensitive information store ayye possibility undi.
Anduke:
Vault
↓
Secret management
separate.
Remote Backend
↓
Terraform state management/security
separate.

🔥 **Ippudu Netflix project motham connect cheyyi**

